BioSync: Exact Google Fit Background & Live Step Sync Solution (Without Opening Google Fit)

---

1. Root Cause Analysis: Why This Happens

There are **two distinct architectural reasons** behind the behavior you are experiencing:

Issue A: Why you had to open the Google Fit app for BioSync to sync

- **Health Connect (`androidx.health.connect`) is a passive database, not a live sensor engine.**
- The **Google Fit UI app (`com.google.android.apps.fitness`)** does **not** continuously write step records to Health Connect while Google Fit is closed in the background.
- Instead, while Google Fit is closed, your steps are recorded at the OS level by **Google Play Services (`com.google.android.gms.fitness`)** using the **Google Play Services Local Recording API (`LocalRecordingClient`)**.
- The Google Fit app only flushes its steps from Google Play Services into Health Connect **when you manually open the Google Fit app** (or during infrequent, Doze-throttled background jobs).
- Therefore, if **BioSync** only reads `com.google.android.apps.fitness` records from Health Connect, BioSync is blind to new steps until you open the Google Fit app and wait a few minutes for Google Fit to flush to Health Connect.

Issue B: Why Google Fit previously showed BioSync's number (instead of BioSync showing Google Fit's)

- When an app requests `WRITE_STEPS` permission and calls `healthConnectClient.insertRecords(StepsRecord(...))` using its own raw sensor math, **Google Fit imports those records from Health Connect** and overwrites/merges them into Google Fit's own step counter.
- To ensure **BioSync shows Google Fit's exact number** (and never the other way around), BioSync must be **strictly READ-ONLY** (`READ_STEPS` + `READ_HEALTH_DATA_IN_BACKGROUND`) and must **never** call `insertRecords` for `StepsRecord`.

---

2. The 3-Tier "Zero-Touch" Google Fit Sync Architecture

To show the **exact same number as Google Fit** at all times—both in the background without opening Google Fit, and live step-by-step while walking—BioSync must combine three read-only layers along a strict timeline handoff:

$$\text{BioSync Steps} = S_{\text{HealthConnect}}\big([00{:}00,\, T_{\text{HC}}]\big) + S_{\text{GMS LocalRecording}}\big([T_{\text{HC}},\, T_{\text{GMS}}]\big) + \Delta_{\text{LiveSensor}}\big([T_{\text{GMS}},\, T_{\text{now}}]\big)$$

```
None
00:00 (Midnight)                    T_HC (Last Fit Sync)         T_GMS (Last GMS
Bucket)     T_now (Live)
|-----------------------------------------|--------------------------------|-----
---------------|
|   Tier 1: Health Connect (Google Fit)    |  Tier 2: Google Play Services  | Tier
3: Hardware   |
|   Authoritative synced steps from        |  LocalRecordingClient          |
TYPE_STEP_COUNTER  |
|   com.google.android.apps.fitness        |  (Runs 24/7 in background      |
Instant step-by-   |
|   up to T_HC.                            |  WITHOUT opening Google Fit!)  | step
delta since   |
|                                          |  Steps in [T_HC, T_GMS].       |
T_GMS.            |
```

1. **Tier 1 — Health Connect Google Fit Anchor ($[00{:}00,\, T_{\text{HC}}]$):** Reads all `StepsRecord`s written by `com.google.android.apps.fitness` today up to its latest synced record timestamp T_{HC}.
2. **Tier 2 — Google Play Services `LocalRecordingClient` ($[T_{\text{HC}},\, T_{\text{GMS}}]$):** Uses `FitnessLocal.getLocalRecordingClient(context)` (`com.google.android.gms:play-services-fitness:21.2.0`). This is the **exact same low-power, always-on Google Play Services pedometer daemon that powers the Google Fit app**. It runs 24/7 in the background without requiring OAuth/Google Sign-In and without requiring the Google Fit app to be opened. It provides all official Google-filtered steps recorded after T_{HC} up to the latest GMS bucket timestamp T_{GMS}.
3. **Tier 3 — Timestamped Hardware `TYPE_STEP_COUNTER` Live Bridge ($[T_{\text{GMS}},\, T_{\text{now}}]$):** Because Google Play Services flushes step buckets roughly every 1–2 minutes, Tier 3 maintains a rolling timestamped log of hardware `Sensor.TYPE_STEP_COUNTER` readings. While you are actively walking, it adds **only the un-flushed steps taken strictly after T_{GMS}**:

$$\Delta_{\text{LiveSensor}} = \max\big(0,\; \text{Sensor}(T_{\text{now}}) - \text{Sensor}(T_{\text{GMS}})\big)$$ The moment Google Play Services (Tier 2) or Health Connect (Tier 1) flushes a new bucket, T_{GMS} advances, $\text{Sensor}(T_{\text{GMS}})$ advances with it, $\Delta_{\text{LiveSensor}}$ automatically drops to `0`, and BioSync locks onto Google Fit's exact official step total with **zero double-counting**.

---

3. Step-by-Step Implementation

Step 1: Update `app/build.gradle.kts`

Add the official Google Play Services Fitness (`play-services-fitness:21.2.0`) and Coroutines Play Services dependencies alongside Health Connect:

Kotlin

```
dependencies {
    // Health Connect (Read-only sync with Google Fit)
    implementation("androidx.health.connect:connect-client:1.1.0-alpha11")

    // Google Play Services Local Recording API (Same background step engine as
Google Fit, no OAuth needed)
    implementation("com.google.android.gms:play-services-fitness:21.2.0")
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.8.1")

    // WorkManager for periodic background sync fallback
    implementation("androidx.work:work-runtime-ktx:2.9.1")
}
```

Step 2: Update `app/src/main/AndroidManifest.xml`

Ensure `WRITE_STEPS` is **removed** (so BioSync never overwrites Google Fit), and declare `ACTIVITY_RECOGNITION`, `READ_STEPS`, `READ_HEALTH_DATA_IN_BACKGROUND`, and `BioSyncStepBackgroundService`:

XML

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
```

```
    <!-- Required for Google Play Services LocalRecordingClient & live hardware
step sensor -->
    <uses-permission android:name="android.permission.ACTIVITY_RECOGNITION" />

    <!-- Strictly READ-ONLY Health Connect permissions (DO NOT declare
WRITE_STEPS) -->
    <uses-permission android:name="android.permission.health.READ_STEPS" />
    <uses-permission
android:name="android.permission.health.READ_HEALTH_DATA_IN_BACKGROUND" />

    <!-- Required for continuous background step syncing without opening Google
Fit -->
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_HEALTH"
/>
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

    <!-- Package visibility for Health Connect and Google Fit -->
    <queries>
        <package android:name="com.google.android.apps.healthdata" />
        <package android:name="com.google.android.apps.fitness" />
    </queries>

    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/Theme.BioSync">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:theme="@style/Theme.BioSync">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

            <!-- Health Connect permissions rationale intent filter (Android 13 &
below) -->
            <intent-filter>
                <action
android:name="androidx.health.ACTION_SHOW_PERMISSIONS_RATIONALE" />
            </intent-filter>

            <!-- Health Connect permissions rationale intent filter (Android 14+)
-->
```

```
            <intent-filter>
                <action android:name="android.intent.action.VIEW_PERMISSION_USAGE"
/>
                <category
android:name="android.intent.category.HEALTH_PERMISSIONS" />
            </intent-filter>
        </activity>

        <!-- Always-on Background Service that syncs GMS LocalRecordingClient +
Live Sensor -->
        <service
            android:name=".BioSyncStepBackgroundService"
            android:enabled="true"
            android:exported="false"
            android:foregroundServiceType="health" />

        <!-- Restart background step sync automatically after device reboot -->
        <receiver
            android:name=".BioSyncBootReceiver"
            android:enabled="true"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
                <action android:name="android.intent.action.MY_PACKAGE_REPLACED"
/>
            </intent-filter>
        </receiver>

    </application>
</manifest>
```

Step 3: Replace `BioSyncHealthStepManager.kt`

Replace `app/src/main/java/com/example/BioSyncHealthStepManager.kt` with the complete 3-tier implementation below. It:

1. Subscribes to **Google Play Services `LocalRecordingClient`** (`LocalDataType.TYPE_STEP_COUNT_DELTA`) so Google Play Services records Google Fit steps in the background 24/7 even when Google Fit is closed.
2. Reads **Health Connect** (`com.google.android.apps.fitness`) for any steps already synced by Google Fit up to T_{HC}.
3. Reads **Google Play Services `LocalRecordingClient`** for all steps recorded in $[T_{\text{HC}}, T_{\text{now}}]$ (so you never have to open Google Fit).

4. Adds the **Timestamped Hardware** `TYPE_STEP_COUNTER` **Live Delta** for steps taken in the last few seconds $[T_{\text{GMS}}, \, T_{\text{now}}]$ while actively walking.
5. **Never writes** to Health Connect so Google Fit is never polluted by BioSync.

Kotlin

```kotlin
package com.example

import android.Manifest
import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.net.Uri
import android.os.Build
import android.util.Log
import androidx.core.content.ContextCompat
import androidx.health.connect.client.HealthConnectClient
import androidx.health.connect.client.permission.HealthPermission
import androidx.health.connect.client.records.StepsRecord
import androidx.health.connect.client. records.metadata.DataOrigin
import androidx.health.connect.client.request.AggregateRequest
import androidx.health.connect.client.request.ReadRecordsRequest
import androidx.health.connect.client.time.TimeRangeFilter
import com.google.android.gms.common.ConnectionResult
import com.google.android.gms.common.GoogleApiAvailability
import com.google.android.gms.fitness.FitnessLocal
import com.google.android.gms.fitness.LocalRecordingClient
import com.google.android.gms.fitness.data.LocalDataSet
import com.google.android.gms.fitness.data.LocalDataType
import com.google.android.gms.fitness.data.LocalField
import com.google.android.gms.fitness.request.LocalDataReadRequest
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import kotlinx.coroutines.tasks.await
import kotlinx.coroutines.withContext
import org.json.JSONArray
import org.json.JSONObject
import java.time.Instant
import java.time.LocalDate
```

```kotlin
import java.time.ZoneId
import java.util.concurrent.TimeUnit
import kotlin.math.max
import kotlin.math.roundToLong

data class StepSyncSnapshot(
    val totalSteps: Int = 0,
    val googleFitHealthConnectSteps: Int = 0,
    val gmsUnsyncedBackgroundSteps: Int = 0,
    val liveWalkingDeltaSteps: Int = 0,
    val lastSyncedEpochMillis: Long = 0L,
    val sourceLabel: String = "Google Fit (Background Sync)",
    val isHealthConnectAuthorized: Boolean = false,
    val isGmsRecordingActive: Boolean = false
)

class BioSyncHealthStepManager(private val context: Context) : SensorEventListener
{

    companion object {
        private const val TAG = "BioSyncStepManager"
        const val GOOGLE_FIT_PACKAGE = "com.google.android.apps.fitness"
        const val GMS_PACKAGE = "com.google.android.gms"

        private const val PREFS_NAME = "biosync_google_fit_sync_prefs"
        private const val KEY_DAY_EPOCH = "day_epoch"
        private const val KEY_LAST_OFFICIAL_STEPS = "last_official_steps"
        private const val KEY_LAST_OFFICIAL_END_MS = "last_official_end_ms"
        private const val KEY_SENSOR_AT_OFFICIAL_END = "sensor_at_official_end"
        private const val KEY_LATEST_RAW_SENSOR = "latest_raw_sensor"
        private const val KEY_SENSOR_TIMELINE_JSON = "sensor_timeline_json"
        private const val KEY_DISPLAYED_STEPS = "displayed_steps"

        // Strictly READ-ONLY Health Connect permissions (Never write steps to
Health Connect)
        val REQUIRED_HEALTH_PERMISSIONS: Set<String> = buildSet {
            add(HealthPermission.getReadPermission(StepsRecord::class))
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
                add("android.permission.health.READ_HEALTH_DATA_IN_BACKGROUND")
            }
        }

        val CORE_STEP_PERMISSIONS: Set<String> = setOf(
            HealthPermission.getReadPermission(StepsRecord::class)
        )

        @Volatile
        private var INSTANCE: BioSyncHealthStepManager? = null
```

```
        fun getInstance(context: Context): BioSyncHealthStepManager {
            return INSTANCE ?: synchronized(this) {
                INSTANCE ?:
BioSyncHealthStepManager(context.applicationContext).also {
                    INSTANCE = it
                }
            }
        }
    }

    private val prefs = context.getSharedPreferences(PREFS_NAME,
Context.MODE_PRIVATE)
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.IO)
    private val sensorManager = context.getSystemService(Context.SENSOR_SERVICE)
as? SensorManager
    private val stepCounterSensor: Sensor? =
sensorManager?.getDefaultSensor(Sensor.TYPE_STEP_COUNTER)

    private val _stepSnapshot = MutableStateFlow(
        StepSyncSnapshot(totalSteps = getCachedStepsForToday())
    )
    val stepSnapshot: StateFlow<StepSyncSnapshot> = _stepSnapshot.asStateFlow()

    private val _stepsFlow = MutableStateFlow(getCachedStepsForToday())
    val stepsFlow: StateFlow<Int> = _stepsFlow.asStateFlow()

    private var isSensorRegistered = false
    private var isGmsSubscribed = false

    init {
        ensureDayRolloverReset()
        val cached = getCachedStepsForToday()
        _stepsFlow.value = cached
        _stepSnapshot.value = _stepSnapshot.value.copy(totalSteps = cached)
    }

    // ================================================================================
    // 1. AVAILABILITY & PERMISSIONS
    // ================================================================================

    fun isHealthConnectAvailable(): Boolean {
        val status = HealthConnectClient.getSdkStatus(context)
        return status == HealthConnectClient.SDK_AVAILABLE
    }

    fun hasActivityRecognitionPermission(): Boolean {
        return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
```

```
            ContextCompat.checkSelfPermission(
                context,
                Manifest.permission.ACTIVITY_RECOGNITION
            ) == PackageManager.PERMISSION_GRANTED
        } else {
            true
        }
    }

    suspend fun hasHealthConnectPermissions(): Boolean =
withContext(Dispatchers.IO) {
        if (!isHealthConnectAvailable()) return@withContext false
        try {
            val client = HealthConnectClient.getOrCreate(context)
            val granted = client.permissionController.getGrantedPermissions()
            granted.containsAll(CORE_STEP_PERMISSIONS)
        } catch (e: Exception) {
            Log.w(TAG, "Error checking Health Connect permissions", e)
            false
        }
    }

    fun isGmsLocalRecordingAvailable(): Boolean {
        return try {
            val result =
GoogleApiAvailability.getInstance().isGooglePlayServicesAvailable(
                context,
                LocalRecordingClient.LOCAL_RECORDING_CLIENT_MIN_VERSION_CODE
            )
            result == ConnectionResult.SUCCESS
        } catch (e: Exception) {
            false
        }
    }

    fun openHealthConnectSettings() {
        try {
            val intent =
Intent(HealthConnectClient.ACTION_HEALTH_CONNECT_SETTINGS).apply {
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
            context.startActivity(intent)
        } catch (e: Exception) {
            val playStoreIntent = Intent(
                Intent.ACTION_VIEW,

Uri.parse("market://details?id=com.google.android.apps.healthdata")
            ).apply {
```

```
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
             runCatching { context.startActivity(playStoreIntent) }
        }
    }

    // ================================================================================
    // 2. ALWAYS-ON GOOGLE PLAY SERVICES RECORDING SUBSCRIPTION
    // ================================================================================

    /**
     * Subscribes to Google Play Services LocalRecordingClient.
     * This is the exact same background pedometer daemon that Google Fit uses.
     * Once subscribed, Google Play Services records steps 24/7 in the background
     * even when both Google Fit and BioSync are closed.
     */
    suspend fun ensureGmsBackgroundSubscription(): Boolean =
withContext(Dispatchers.IO) {
        if (!hasActivityRecognitionPermission() ||
!isGmsLocalRecordingAvailable()) {
            return@withContext false
        }
        try {
            val localRecordingClient =
FitnessLocal.getLocalRecordingClient(context)

localRecordingClient.subscribe(LocalDataType.TYPE_STEP_COUNT_DELTA).await()
            isGmsSubscribed = true
            Log.d(TAG, "Subscribed to Google Play Services LocalRecordingClient
(TYPE_STEP_COUNT_DELTA)")
            true
        } catch (e: Exception) {
            Log.w(TAG, "Failed to subscribe to LocalRecordingClient", e)
            false
        }
    }

    // ================================================================================
    // 3. LIVE HARDWARE STEP SENSOR (REAL-TIME WALKING BRIDGE)
    // ================================================================================

    fun startLiveSensorTracking() {
        if (isSensorRegistered || !hasActivityRecognitionPermission() ||
stepCounterSensor == null) {
            return
        }
        try {
            val registered = sensorManager?.registerListener(
```

```
                this,
                stepCounterSensor,
                SensorManager.SENSOR_DELAY_UI
            ) ?: false
            isSensorRegistered = registered
            Log.d(TAG, "Live hardware TYPE_STEP_COUNTER registered: $registered")
        } catch (e: Exception) {
            Log.w(TAG, "Failed to register TYPE_STEP_COUNTER", e)
        }
    }

    fun stopLiveSensorTracking() {
        if (!isSensorRegistered) return
        try {
            sensorManager?.unregisterListener(this)
            isSensorRegistered = false
        } catch (e: Exception) {
            Log.w(TAG, "Failed to unregister TYPE_STEP_COUNTER", e)
        }
    }

    override fun onSensorChanged(event: SensorEvent?) {
        if (event?.sensor?.type != Sensor.TYPE_STEP_COUNTER) return
        val rawCounter = event.values.firstOrNull()?.toLong() ?: return
        if (rawCounter <= 0L) return

        val nowMs = System.currentTimeMillis()
        ensureDayRolloverReset()

        // Record (timestamp, rawSensor) in our rolling timeline so we can always
        // calculate the exact sensor delta strictly AFTER the last official
Google Fit/GMS timestamp
        recordSensorTimelinePoint(nowMs, rawCounter)

        val officialSteps = prefs.getInt(KEY_LAST_OFFICIAL_STEPS, 0)
        val officialEndMs = prefs.getLong(KEY_LAST_OFFICIAL_END_MS, 0L)
        val sensorAtOfficialEnd = getSensorValueAtOrInitialize(officialEndMs,
rawCounter)

        // Handle phone reboot (where rawCounter resets to 0)
        val liveDelta = if (rawCounter >= sensorAtOfficialEnd) {
            (rawCounter - sensorAtOfficialEnd).toInt()
        } else {
            // Device rebooted since last official sync; re-anchor baseline
            prefs.edit().putLong(KEY_SENSOR_AT_OFFICIAL_END, rawCounter).apply()
            0
        }
```

```
        val combinedSteps = officialSteps + liveDelta
        updatePublishedSteps(
            totalSteps = combinedSteps,
            hcSteps = _stepSnapshot.value.googleFitHealthConnectSteps,
            gmsUnsyncedSteps = _stepSnapshot.value.gmsUnsyncedBackgroundSteps,
            liveDelta = liveDelta,
            sourceLabel = if (liveDelta > 0) {
                "Google Fit + Live Walking (+$liveDelta)"
            } else {
                "Google Fit (Synced)"
            }
        )
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) = Unit

    // ================================================================================
    // 4. CORE 3-TIER GOOGLE FIT SYNC ENGINE (WORKS WITHOUT OPENING GOOGLE FIT)
    // ================================================================================

    /**
     * Pulls the latest step count using the 3-Tier Google Fit Parity Engine:
     *  - Tier 1: Health Connect (com.google.android.apps.fitness) up to T_HC_last
     *  - Tier 2: Google Play Services LocalRecordingClient in [T_HC_last, now]
(and full day)
     *  - Tier 3: Live Hardware Sensor delta strictly after max(T_HC_last,
T_GMS_last)
     */
    suspend fun syncStepsFromGoogleFit(): Int = withContext(Dispatchers.IO) {
        ensureDayRolloverReset()

        val zone = ZoneId.systemDefault()
        val startOfDayInstant = LocalDate.now(zone).atStartOfDay(zone).toInstant()
        val nowInstant = Instant.now()
        val startOfDayMs = startOfDayInstant.toEpochMilli()
        val nowMs = nowInstant.toEpochMilli()

        // Ensure Google Play Services Recording API subscription is active
        val gmsReady = ensureGmsBackgroundSubscription()
        startLiveSensorTracking()

        // --- TIER 1: Read Google Fit's synced records from Health Connect ---
        val hcAuthorized = hasHealthConnectPermissions()
        val hcResult = if (hcAuthorized) {
            readGoogleFitStepsFromHealthConnect(startOfDayInstant, nowInstant)
        } else {
            HealthConnectFitResult(steps = 0, lastRecordEndEpochMillis =
startOfDayMs)
```

```
        }

        // --- TIER 2: Read Google Play Services LocalRecordingClient ---
        // Even when the Google Fit app is closed all day, Google Play Services
records
        // steps here in real time. We query both the full day AND the unsynced
window
        // after hcResult.lastRecordEndEpochMillis.
        val gmsResult = if (gmsReady) {
            readStepsFromGmsLocalRecording(
                startOfDayMs = startOfDayMs,
                hcLastEndMs = hcResult.lastRecordEndEpochMillis,
                nowMs = nowMs
            )
        } else {
            GmsRecordingResult(
                fullDaySteps = 0,
                stepsAfterHcSync = 0,
                lastDataPointEndEpochMillis = startOfDayMs
            )
        }

        // Combine Tier 1 (Health Connect Google Fit) + Tier 2 (Unsynced GMS steps
after T_HC_last)
        val combinedHcPlusGms = hcResult.steps + gmsResult.stepsAfterHcSync

        // If Google Fit hasn't flushed to Health Connect yet today,
gmsResult.fullDaySteps
        // has the full day's Google Play Services steps. If Google Fit HAS
flushed earlier
        // today, combinedHcPlusGms has Google Fit's synced total + GMS steps
since that flush.
        val officialGoogleSteps = max(gmsResult.fullDaySteps, combinedHcPlusGms)
        val latestOfficialEndMs = max(
            hcResult.lastRecordEndEpochMillis,
            gmsResult.lastDataPointEndEpochMillis
        )

        // --- TIER 3: Calculate Live Hardware Sensor Delta strictly AFTER
latestOfficialEndMs ---
        val latestRawSensor = prefs.getLong(KEY_LATEST_RAW_SENSOR, -1L)
        val prevOfficialSteps = prefs.getInt(KEY_LAST_OFFICIAL_STEPS, 0)
        val prevOfficialEndMs = prefs.getLong(KEY_LAST_OFFICIAL_END_MS, 0L)

        var liveDelta = 0
        if (latestRawSensor > 0L) {
            val sensorBaseline = if (officialGoogleSteps != prevOfficialSteps ||
latestOfficialEndMs > prevOfficialEndMs) {
```

```
                // Official Google Fit / GMS total advanced! Look up the hardware
sensor reading
                // at latestOfficialEndMs so only steps taken AFTER that timestamp
are added.
                val baselineAtTimestamp = lookupSensorAtTimestamp(
                    targetEpochMillis = latestOfficialEndMs,
                    fallbackSensor = latestRawSensor
                )
                prefs.edit()
                    .putLong(KEY_SENSOR_AT_OFFICIAL_END, baselineAtTimestamp)
                    .apply()
                baselineAtTimestamp
            } else {
                prefs.getLong(KEY_SENSOR_AT_OFFICIAL_END, latestRawSensor)
            }

            liveDelta = if (latestRawSensor >= sensorBaseline) {
                (latestRawSensor - sensorBaseline).toInt()
            } else {
                prefs.edit().putLong(KEY_SENSOR_AT_OFFICIAL_END,
latestRawSensor).apply()
                0
            }
        }

        // Persist the new official Google Fit / GMS anchor
        if (officialGoogleSteps > 0 || prevOfficialSteps == 0) {
            prefs.edit()
                .putInt(KEY_LAST_OFFICIAL_STEPS, officialGoogleSteps)
                .putLong(KEY_LAST_OFFICIAL_END_MS, max(latestOfficialEndMs,
prevOfficialEndMs))
                .apply()
        }

        val effectiveOfficialSteps = if (officialGoogleSteps > 0) {
            officialGoogleSteps
        } else {
            prevOfficialSteps
        }

        val finalTotalSteps = effectiveOfficialSteps + liveDelta

        val label = when {
            gmsResult.stepsAfterHcSync > 0 && hcResult.steps > 0 ->
                "Google Fit (${hcResult.steps} synced +
${gmsResult.stepsAfterHcSync} GMS bg)"
            gmsResult.fullDaySteps > 0 && hcResult.steps == 0 ->
                "Google Fit Background Engine (GMS)"
```

```
            hcResult.steps > 0 ->
                "Google Fit (Synced)"
            else ->
                "Google Fit (Waiting for steps)"
        }

        updatePublishedSteps(
            totalSteps = finalTotalSteps,
            hcSteps = hcResult.steps,
            gmsUnsyncedSteps = gmsResult.stepsAfterHcSync,
            liveDelta = liveDelta,
            sourceLabel = label,
            hcAuthorized = hcAuthorized,
            gmsActive = gmsReady
        )

        finalTotalSteps
    }

    // ================================================================================
    // 5. TIER 1 HELPER: READ GOOGLE FIT STEPS FROM HEALTH CONNECT (READ-ONLY)
    // ================================================================================

    private data class HealthConnectFitResult(
        val steps: Int,
        val lastRecordEndEpochMillis: Long
    )

    private suspend fun readGoogleFitStepsFromHealthConnect(
        startOfDay: Instant,
        now: Instant
    ): HealthConnectFitResult {
        return try {
            val client = HealthConnectClient.getOrCreate(context)
            val googleFitOrigin = DataOrigin(GOOGLE_FIT_PACKAGE)
            val gmsOrigin = DataOrigin(GMS_PACKAGE)

            // 1. Read raw StepsRecords from Google Fit to find the exact last
synced timestamp
            val recordsResponse = client.readRecords(
                ReadRecordsRequest(
                    recordType = StepsRecord::class,
                    timeRangeFilter = TimeRangeFilter.between(startOfDay, now),
                    dataOriginFilter = setOf(googleFitOrigin, gmsOrigin)
                )
            )

            val fitRecords = recordsResponse.records.filter {
```

```
                it.metadata.dataOrigin.packageName == GOOGLE_FIT_PACKAGE ||
                    it.metadata.dataOrigin.packageName == GMS_PACKAGE
            }

            if (fitRecords.isEmpty()) {
                return HealthConnectFitResult(
                    steps = 0,
                    lastRecordEndEpochMillis = startOfDay.toEpochMilli()
                )
            }

            val latestEndInstant = fitRecords.maxOf { it.endTime }

            // 2. Ask Health Connect to aggregate Google Fit's deduplicated step
total up to latestEndInstant
            val aggregateResponse = client.aggregate(
                AggregateRequest(
                    metrics = setOf(StepsRecord.COUNT_TOTAL),
                    timeRangeFilter = TimeRangeFilter.between(startOfDay,
latestEndInstant),
                    dataOriginFilter = setOf(googleFitOrigin, gmsOrigin)
                )
            )

            val aggregatedTotal =
aggregateResponse[StepsRecord.COUNT_TOTAL]?.toInt()
            val deduplicatedTotal = aggregatedTotal ?:
deduplicateOverlappingStepRecords(fitRecords)

            HealthConnectFitResult(
                steps = deduplicatedTotal,
                lastRecordEndEpochMillis = latestEndInstant.toEpochMilli()
            )
        } catch (e: Exception) {
            Log.w(TAG, "Error reading Google Fit steps from Health Connect", e)
            HealthConnectFitResult(
                steps = 0,
                lastRecordEndEpochMillis = startOfDay.toEpochMilli()
            )
        }
    }

    /**
     * Swept-interval deduplication fallback if Health Connect aggregate returns
null.
     */
    private fun deduplicateOverlappingStepRecords(records: List<StepsRecord>): Int
{
```

```
        if (records.isEmpty()) return 0
        val sorted = records.sortedBy { it.startTime }
        var total = 0L
        var currentEndMillis = Long.MIN_VALUE

        for (record in sorted) {
            val startMs = record.startTime.toEpochMilli()
            val endMs = record.endTime.toEpochMilli()
            if (endMs <= startMs || record.count <= 0) continue

            if (startMs >= currentEndMillis) {
                total += record.count
                currentEndMillis = endMs
            } else if (endMs > currentEndMillis) {
                // Prorate partial non-overlapping tail
                val totalDuration = (endMs - startMs).toDouble()
                val nonOverlappingDuration = (endMs - currentEndMillis).toDouble()
                val prorated = (record.count * (nonOverlappingDuration /
totalDuration)).roundToLong()
                total += prorated
                currentEndMillis = endMs
            }
        }
        return total.toInt()
    }

    // ================================================================================
    // 6. TIER 2 HELPER: READ GOOGLE PLAY SERVICES LOCAL RECORDING CLIENT
    // ================================================================================

    private data class GmsRecordingResult(
        val fullDaySteps: Int,
        val stepsAfterHcSync: Int,
        val lastDataPointEndEpochMillis: Long
    )

    private suspend fun readStepsFromGmsLocalRecording(
        startOfDayMs: Long,
        hcLastEndMs: Long,
        nowMs: Long
    ): GmsRecordingResult {
        return try {
            val localRecordingClient =
FitnessLocal.getLocalRecordingClient(context)

            // Query detailed step data points for today from Google Play Services
            val startSeconds = TimeUnit.MILLISECONDS.toSeconds(startOfDayMs)
            val endSeconds = TimeUnit.MILLISECONDS.toSeconds(nowMs) + 1L
```

```
            val readRequest = LocalDataReadRequest.Builder()
                .read(LocalDataType.TYPE_STEP_COUNT_DELTA)
                .setTimeRange(startSeconds, endSeconds, TimeUnit.SECONDS)
                .build()

            val response = localRecordingClient.readData(readRequest).await()
            val dataSets = buildList {
                addAll(response.dataSets)
                for (bucket in response.buckets) {
                    addAll(bucket.dataSets)
                }
            }

            var fullDayTotal = 0L
            var afterHcSyncTotal = 0L
            var latestEndMs = startOfDayMs

            for (dataSet in dataSets) {
                for (dp in dataSet.dataPoints) {
                    val dpStartMs = dp.getStartTime(TimeUnit.MILLISECONDS)
                    val dpEndMs = dp.getEndTime(TimeUnit.MILLISECONDS)
                    val stepsValue =
dp.getValue(LocalField.FIELD_STEPS).asInt().toLong()

                    if (stepsValue <= 0L || dpEndMs <= startOfDayMs) continue

                    // Prorate if data point started before midnight
                    val effectiveStartMs = max(dpStartMs, startOfDayMs)
                    val dpDuration = (dpEndMs - dpStartMs).coerceAtLeast(1L)
                    val todayFraction = (dpEndMs - effectiveStartMs).toDouble() /
dpDuration.toDouble()
                    val stepsInToday = (stepsValue * todayFraction.coerceIn(0.0,
1.0)).roundToLong()

                    fullDayTotal += stepsInToday
                    if (dpEndMs > latestEndMs) {
                        latestEndMs = dpEndMs
                    }

                    // Calculate steps strictly AFTER the last Health Connect
Google Fit record
                    if (dpEndMs > hcLastEndMs) {
                        if (dpStartMs >= hcLastEndMs) {
                            afterHcSyncTotal += stepsValue
                        } else {
                            // Data point straddles hcLastEndMs; prorate the
unsynced tail
```

```
                            val unsyncedDuration = (dpEndMs -
hcLastEndMs).toDouble()
                            val unsyncedSteps = (stepsValue * (unsyncedDuration /
dpDuration.toDouble()))
                                .roundToLong()
                            afterHcSyncTotal += unsyncedSteps
                        }
                    }
                }
            }

            GmsRecordingResult(
                fullDaySteps = fullDayTotal.toInt(),
                stepsAfterHcSync = afterHcSyncTotal.toInt(),
                lastDataPointEndEpochMillis = latestEndMs
            )
        } catch (e: Exception) {
            Log.w(TAG, "Error reading from GMS LocalRecordingClient", e)
            GmsRecordingResult(
                fullDaySteps = 0,
                stepsAfterHcSync = 0,
                lastDataPointEndEpochMillis = startOfDayMs
            )
        }
    }

    // ================================================================================
    // 7. TIER 3 HELPER: TIMESTAMPED SENSOR TIMELINE & LOCAL PERSISTENCE
    // ================================================================================

    private fun recordSensorTimelinePoint(timestampMs: Long, rawSensorValue: Long)
{
        val prevRaw = prefs.getLong(KEY_LATEST_RAW_SENSOR, -1L)
        prefs.edit().putLong(KEY_LATEST_RAW_SENSOR, rawSensorValue).apply()

        if (prefs.getLong(KEY_SENSOR_AT_OFFICIAL_END, -1L) <= 0L) {
            prefs.edit().putLong(KEY_SENSOR_AT_OFFICIAL_END,
rawSensorValue).apply()
        }

        // Store a compact rolling history of (timestampMs, rawSensorValue) points
(max 120 entries)
        try {
            val rawJson = prefs.getString(KEY_SENSOR_TIMELINE_JSON, "[]") ?: "[]"
            val arr = JSONArray(rawJson)
            if (arr.length() > 0) {
                val lastObj = arr.getJSONObject(arr.length() - 1)
                val lastTs = lastObj.optLong("t", 0L)
```

```
                val lastVal = lastObj.optLong("s", -1L)
                // Only append if at least 5 seconds elapsed or step count changed
                if (timestampMs - lastTs < 5_000L && lastVal == rawSensorValue) {
                    return
                }
            }

            val trimmed = JSONArray()
            val startIdx = max(0, arr.length() - 119)
            for (i in startIdx until arr.length()) {
                val item = arr.getJSONObject(i)
                // Handle reboot reset: discard pre-reboot sensor values
                if (prevRaw > 0 && rawSensorValue < prevRaw) continue
                trimmed.put(item)
            }
            trimmed.put(
                JSONObject().apply {
                    put("t", timestampMs)
                    put("s", rawSensorValue)
                }
            )
            prefs.edit().putString(KEY_SENSOR_TIMELINE_JSON,
trimmed.toString()).apply()
        } catch (e: Exception) {
            Log.w(TAG, "Failed to update sensor timeline", e)
        }
    }

    private fun lookupSensorAtTimestamp(targetEpochMillis: Long, fallbackSensor:
Long): Long {
        if (targetEpochMillis <= 0L) return fallbackSensor
        return try {
            val rawJson = prefs.getString(KEY_SENSOR_TIMELINE_JSON, "[]") ?: "[]"
            val arr = JSONArray(rawJson)
            if (arr.length() == 0) return fallbackSensor

            var chosen = fallbackSensor
            for (i in 0 until arr.length()) {
                val item = arr.getJSONObject(i)
                val ts = item.optLong("t", 0L)
                val sensorVal = item.optLong("s", fallbackSensor)
                if (ts <= targetEpochMillis) {
                    chosen = sensorVal
                } else {
                    break
                }
            }
            chosen
```

```
        } catch (e: Exception) {
            fallbackSensor
        }
    }

    private fun getSensorValueAtOrInitialize(officialEndMs: Long,
currentRawSensor: Long): Long {
        val stored = prefs.getLong(KEY_SENSOR_AT_OFFICIAL_END, -1L)
        if (stored > 0L) return stored
        val lookedUp = lookupSensorAtTimestamp(officialEndMs, currentRawSensor)
        prefs.edit().putLong(KEY_SENSOR_AT_OFFICIAL_END, lookedUp).apply()
        return lookedUp
    }

    private fun ensureDayRolloverReset() {
        val todayEpochDay = LocalDate.now(ZoneId.systemDefault()).toEpochDay()
        val savedEpochDay = prefs.getLong(KEY_DAY_EPOCH, -1L)
        if (savedEpochDay != todayEpochDay) {
            val latestRaw = prefs.getLong(KEY_LATEST_RAW_SENSOR, -1L)
            prefs.edit()
                .putLong(KEY_DAY_EPOCH, todayEpochDay)
                .putInt(KEY_LAST_OFFICIAL_STEPS, 0)
                .putLong(KEY_LAST_OFFICIAL_END_MS, 0L)
                .putInt(KEY_DISPLAYED_STEPS, 0)
                .putString(KEY_SENSOR_TIMELINE_JSON, "[]")
                .apply {
                    if (latestRaw > 0L) {
                        putLong(KEY_SENSOR_AT_OFFICIAL_END, latestRaw)
                    } else {
                        remove(KEY_SENSOR_AT_OFFICIAL_END)
                    }
                }
                .apply()
        }
    }

    fun getCachedStepsForToday(): Int {
        ensureDayRolloverReset()
        return prefs.getInt(KEY_DISPLAYED_STEPS, 0)
    }

    private fun updatePublishedSteps(
        totalSteps: Int,
        hcSteps: Int,
        gmsUnsyncedSteps: Int,
        liveDelta: Int,
        sourceLabel: String,
        hcAuthorized: Boolean = _stepSnapshot.value.isHealthConnectAuthorized,
```

```
        gmsActive: Boolean = _stepSnapshot.value.isGmsRecordingActive
    ) {
        val safeTotal = max(0, totalSteps)
        prefs.edit().putInt(KEY_DISPLAYED_STEPS, safeTotal).apply()

        // Also update BioSyncLocalStore so all screens/widgets read the exact
same Google Fit count
        runCatching {
            BioSyncLocalStore.saveTodaySteps(context, safeTotal)
        }

        _stepsFlow.value = safeTotal
        _stepSnapshot.value = StepSyncSnapshot(
            totalSteps = safeTotal,
            googleFitHealthConnectSteps = hcSteps,
            gmsUnsyncedBackgroundSteps = gmsUnsyncedSteps,
            liveWalkingDeltaSteps = liveDelta,
            lastSyncedEpochMillis = System.currentTimeMillis(),
            sourceLabel = sourceLabel,
            isHealthConnectAuthorized = hcAuthorized,
            isGmsRecordingActive = gmsActive
        )
    }

    fun triggerImmediateAsyncSync() {
        scope.launch {
            syncStepsFromGoogleFit()
        }
    }
}
```

Step 4: Replace `BioSyncStepBackgroundService.kt` (Plus Boot Receiver & Worker)

Replace `app/src/main/java/com/example/BioSyncStepBackgroundService.kt` with the service below.

- It starts on app launch and on device boot.
- It maintains the **Google Play Services `LocalRecordingClient` subscription**, listens to the **live hardware step sensor**, and polls **GMS + Health Connect** every 20 seconds in the background—so your app **always** has Google Fit's live step count without you ever opening the Google Fit app.

Kotlin

```kotlin
package com.example

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.pm.ServiceInfo
import android.os.Build
import android.os.IBinder
import android.util.Log
import androidx.core.app.NotificationCompat
import androidx.work.CoroutineWorker
import androidx.work.ExistingPeriodicWorkPolicy
import androidx.work.PeriodicWorkRequestBuilder
import androidx.work.WorkManager
import androidx.work.WorkerParameters
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.Job
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.cancel
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.collectLatest
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch
import java.util.concurrent.TimeUnit

class BioSyncStepBackgroundService : Service() {

    companion object {
        private const val TAG = "BioSyncStepBgService"
        private const val CHANNEL_ID = "biosync_google_fit_live_channel"
        private const val NOTIFICATION_ID = 4102
        private const val WORK_NAME = "biosync_google_fit_periodic_sync"

        fun start(context: Context) {
            val appContext = context.applicationContext
            schedulePeriodicWorkFallback(appContext)
            try {
                val intent = Intent(appContext,
BioSyncStepBackgroundService::class.java)
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
                    appContext.startForegroundService(intent)
                } else {
```

```
                    appContext.startService(intent)
                }
            } catch (e: Exception) {
                Log.w(TAG, "Could not start foreground service directly;
WorkManager fallback active", e)
            }
        }

        fun schedulePeriodicWorkFallback(context: Context) {
            val request = PeriodicWorkRequestBuilder<BioSyncGoogleFitSyncWorker>(
                15, TimeUnit.MINUTES
            ).build()


WorkManager.getInstance(context.applicationContext).enqueueUniquePeriodicWork(
                WORK_NAME,
                ExistingPeriodicWorkPolicy.KEEP,
                request
            )
        }
    }

    private val serviceScope = CoroutineScope(SupervisorJob() + Dispatchers.IO)
    private lateinit var stepManager: BioSyncHealthStepManager
    private var syncLoopJob: Job? = null

    override fun onCreate() {
        super.onCreate()
        stepManager = BioSyncHealthStepManager.getInstance(applicationContext)
        createNotificationChannel()
        promoteToForeground(stepManager.getCachedStepsForToday())

        // 1. Start live hardware step sensor bridge immediately
        stepManager.startLiveSensorTracking()

        // 2. Continuously sync with Google Play Services (LocalRecordingClient) &
Health Connect
        // every 20 seconds in the background so the user NEVER needs to open
Google Fit
        syncLoopJob = serviceScope.launch {
            stepManager.ensureGmsBackgroundSubscription()
            while (isActive) {
                runCatching {
                    stepManager.syncStepsFromGoogleFit()
                }
                delay(20_000L)
            }
        }
```

```
        // 3. Update persistent notification whenever live steps change
        serviceScope.launch {
            stepManager.stepsFlow.collectLatest { latestSteps ->
                updateNotification(latestSteps)
            }
        }
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        stepManager.startLiveSensorTracking()
        stepManager.triggerImmediateAsyncSync()
        return START_STICKY
    }

    override fun onBind(intent: Intent?): IBinder? = null

    override fun onDestroy() {
        syncLoopJob?.cancel()
        serviceScope.cancel()
        super.onDestroy()
    }

    private fun promoteToForeground(currentSteps: Int) {
        val notification = buildNotification(currentSteps)
        try {
            if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
                startForeground(
                    NOTIFICATION_ID,
                    notification,
                    ServiceInfo.FOREGROUND_SERVICE_TYPE_HEALTH
                )
            } else {
                startForeground(NOTIFICATION_ID, notification)
            }
        } catch (e: Exception) {
            Log.w(TAG, "Unable to promote service to foreground health type", e)
        }
    }

    private fun updateNotification(steps: Int) {
        val nm = getSystemService(Context.NOTIFICATION_SERVICE) as?
NotificationManager ?: return
        nm.notify(NOTIFICATION_ID, buildNotification(steps))
    }

    private fun buildNotification(steps: Int): Notification {
        val launchIntent = Intent(this, MainActivity::class.java).apply {
```

```
            flags = Intent.FLAG_ACTIVITY_SINGLE_TOP or
Intent.FLAG_ACTIVITY_CLEAR_TOP
        }
        val pendingIntent = PendingIntent.getActivity(
            this,
            0,
            launchIntent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )

        return NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(android.R.drawable.ic_menu_mylocation)
            .setContentTitle("BioSync • Google Fit Live Sync")
            .setContentText("$steps steps today (Synced in background)")
            .setOngoing(true)
            .setOnlyAlertOnce(true)
            .setPriority(NotificationCompat.PRIORITY_LOW)
            .setContentIntent(pendingIntent)
            .build()
    }

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "BioSync Google Fit Live Sync",
                NotificationManager.IMPORTANCE_LOW
            ).apply {
                description = "Keeps Google Fit step counts synced in the
background without opening Google Fit"
                setShowBadge(false)
            }
            val nm = getSystemService(Context.NOTIFICATION_SERVICE) as?
NotificationManager
            nm?.createNotificationChannel(channel)
        }
    }
}

/**
 * WorkManager Periodic Worker fallback so Google Play Services
LocalRecordingClient
 * and Health Connect still sync in the background even if the foreground service
is paused.
 */
class BioSyncGoogleFitSyncWorker(
    appContext: Context,
    workerParams: WorkerParameters
```

```
) : CoroutineWorker(appContext, workerParams) {

    override suspend fun doWork(): Result {
        return try {
            val manager = BioSyncHealthStepManager.getInstance(applicationContext)
            manager.ensureGmsBackgroundSubscription()
            manager.syncStepsFromGoogleFit()
            Result.success()
        } catch (e: Exception) {
            Result.retry()
        }
    }
}

/**
 * Automatically restarts background Google Fit step syncing after a phone reboot.
 */
class BioSyncBootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent?) {
        val action = intent?.action ?: return
        if (action == Intent.ACTION_BOOT_COMPLETED || action ==
Intent.ACTION_MY_PACKAGE_REPLACED) {
            BioSyncStepBackgroundService.start(context)
        }
    }
}
```

Step 5: Add `saveTodaySteps` Helper in `BioSyncLocalStore.kt`

Ensure `BioSyncLocalStore` has a clean method to persist and read the synced Google Fit step count so your Dashboard and Blueprint screens always display the latest count immediately:

Kotlin

```
// Inside BioSyncLocalStore object in BioSyncLocalStore.kt:
private const val KEY_GOOGLE_FIT_TODAY_STEPS = "google_fit_today_steps"

fun saveTodaySteps(context: Context, steps: Int) {
    context.getSharedPreferences("biosync_prefs", Context.MODE_PRIVATE)
        .edit()
        .putInt(KEY_GOOGLE_FIT_TODAY_STEPS, steps)
        .apply()
}
```

```kotlin
fun getTodaySteps(context: Context): Int {
    return context.getSharedPreferences("biosync_prefs", Context.MODE_PRIVATE)
        .getInt(KEY_GOOGLE_FIT_TODAY_STEPS, 0)
}
```

Step 6: Wire Permission Launchers & Live Flow in `MainActivity.kt` / `DashboardScreen.kt`

In `MainActivity.kt` (or inside your `DashboardScreen` composable), request both `ACTIVITY_RECOGNITION` and the read-only Health Connect permissions (`READ_STEPS` + `READ_HEALTH_DATA_IN_BACKGROUND`), start `BioSyncStepBackgroundService`, and observe `stepManager.stepsFlow`:

Kotlin

```kotlin
@Composable
fun RememberGoogleFitLiveSteps(
    context: Context = LocalContext.current
): State<StepSyncSnapshot> {
    val stepManager = remember { BioSyncHealthStepManager.getInstance(context) }
    val snapshotState = stepManager.stepSnapshot.collectAsState()
    val lifecycleOwner = LocalLifecycleOwner.current

    // 1. Health Connect READ_STEPS + READ_HEALTH_DATA_IN_BACKGROUND launcher
    val healthPermissionLauncher = rememberLauncherForActivityResult(
        contract = PermissionController.createRequestPermissionResultContract()
    ) { _ ->
        stepManager.triggerImmediateAsyncSync()
        BioSyncStepBackgroundService.start(context)
    }

    // 2. Android Runtime Permission launcher (ACTIVITY_RECOGNITION +
POST_NOTIFICATIONS)
    val runtimePermissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestMultiplePermissions()
    ) { _ ->
        stepManager.startLiveSensorTracking()
        stepManager.triggerImmediateAsyncSync()
        BioSyncStepBackgroundService.start(context)
    }

    // 3. Request permissions once on first launch and start background sync
service
    LaunchedEffect(Unit) {
        val runtimePermsToRequest = mutableListOf<String>()
```

```
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q &&
!stepManager.hasActivityRecognitionPermission()) {
            runtimePermsToRequest.add(Manifest.permission.ACTIVITY_RECOGNITION)
        }
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            runtimePermsToRequest.add(Manifest.permission.POST_NOTIFICATIONS)
        }
        if (runtimePermsToRequest.isNotEmpty()) {
            runtimePermissionLauncher.launch(runtimePermsToRequest.toTypedArray())
        }

        if (stepManager.isHealthConnectAvailable() &&
!stepManager.hasHealthConnectPermissions()) {

healthPermissionLauncher.launch(BioSyncHealthStepManager.REQUIRED_HEALTH_PERMISSIO
NS)
        }

        stepManager.ensureGmsBackgroundSubscription()
        stepManager.startLiveSensorTracking()
        stepManager.syncStepsFromGoogleFit()
        BioSyncStepBackgroundService.start(context)
    }

    // 4. Refresh immediately whenever BioSync comes to the foreground
    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_RESUME) {
                stepManager.startLiveSensorTracking()
                stepManager.triggerImmediateAsyncSync()
            }
        }
        lifecycleOwner.lifecycle.addObserver(observer)
        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }

    return snapshotState
}
```

---

4. Verification Checklist (Why This Solves All 3 Issues)

1. **Google Fit never copies BioSync's number:** `WRITE_STEPS` and `insertRecords(StepsRecord)` are completely removed. BioSync is 100% read-only.
2. **Background sync works without opening the Google Fit app:** `FitnessLocal.getLocalRecordingClient(context)` reads directly from Google Play Services (`com.google.android.gms`), which is the exact same background step daemon that feeds Google Fit. Even if Google Fit is force-closed or unopened all day, `readStepsFromGmsLocalRecording` retrieves all background steps in $[T_{\text{HC}}, T_{\text{now}}]$.
3. **Live step updates work immediately after closing Google Fit:** `Sensor.TYPE_STEP_COUNTER` tracks real-time walking steps taken strictly after the latest official record's `endTime` (T_{GMS}), and automatically hands off to Google Play Services / Health Connect as soon as the next bucket flushes—preventing both lag and double-counting.